TECHNICAL ASSIGNMENT SUBMISSION REPORT

Senior DevOps Engineer – Technical Assignment Submission Report

Cloud-Agnostic, Automated, Zero-Downtime Deployment Platform

Candidate	Girish Burade
Role	Senior DevOps Engineer
Repository	github.com/Girish7010/cloud-agnostic-zero-downtime-platform
Submission date	4 November 2026
Final CI run	GitHub Actions run #5 (41810479665), all 5 jobs successful

Scope at a glance. Every runtime test in this report ran on a **local single-node Kubernetes (kind) cluster** using a **reference application** written to the assignment's black-box contract. Cloud infrastructure is implemented as validated Terraform but was **not provisioned**. Limitations are listed in full in section 10.

1. Executive summary

The assignment asked for a platform that provisions infrastructure on any cloud, deploys a stateless API and a queue worker through a multi-stage pipeline without downtime, scales to 15 times its baseline load, and supports both a shared environment and dedicated per-tenant environments while keeping EU data in the EU.

What was delivered. All four deliverables in the assignment brief:

- a Git repository with README, infrastructure as code, pipelines and scripts;
- a five-page architecture document;
- test evidence;
- a one-page change request.

Main objectives addressed

- Build the application image once, then scan, sign and promote the *same* image through dev, staging and production.
- Release new versions while traffic continues, and roll back automatically when a release is unhealthy.
- Scale the API on CPU and the workers on queue depth without losing jobs.
- Create a new tenant from one reviewed change, and keep EU tenants on EU infrastructure.

Key highlights, all backed by raw logs in the repository

- **Deploy under load:** a new version was rolled out during continuous traffic, with **36,001 requests and 0 failed**.
- **Automatic rollback:** two deliberately bad releases were detected and rolled back without manual action, once through a metric gate and once through a readiness check.
- **15x ratio load test:** traffic went from 10 to 150 requests per second and back, with 0 failed requests and **1,170 of 1,170 background jobs completed**.
- **Backup and restore:** the database was restored into a new database, and the restored data matched the original exactly.
- **CI/CD pipeline:** GitHub Actions run #5 passed all five stages: test, build, vulnerability scan, SBOM, signing and approved promotion.

`Tested` ran, with evidence `Implemented` code exists and validates, not run on real infrastructure `Design` documented only `Not done`

2. Architecture overview

The platform has three layers. Each layer depends only on a small, fixed set of outputs from the layer below (cluster endpoint, database and Redis addresses, secret references, region, data-residency flag). As a result, changing cloud provider means writing one new provider module; the platform and application layers stay the same.

Component	Technology	Purpose	Status
Application (reference)	Python 3.13, FastAPI, Redis Streams	API on port 8080 with `/healthz`, `/readyz`, `/metrics`; worker processing jobs of up to 60 s	`Tested` locally
Container image	Docker, GitHub Container Registry	One image for API and worker, always deployed by its immutable digest	`Tested` in CI
Orchestration	Kubernetes (kind locally), Helm 4	One chart for shared and dedicated tenants; probes, disruption budgets, graceful shutdown	`Tested` locally

Component	Technology	Purpose	Status
Autoscaling	HPA (CPU), KEDA (queue lag)	API and worker scale independently	**Tested** locally
Infrastructure	Terraform, AWS (EKS, RDS, ElastiCache Valkey, KMS, Secrets Manager)	Provider layer for the cloud target	**Implemented**, not applied
GitOps and canary	Argo CD, Argo Rollouts, Prometheus	Pull-based deployment and canary analysis in the cloud	**Design** / config only

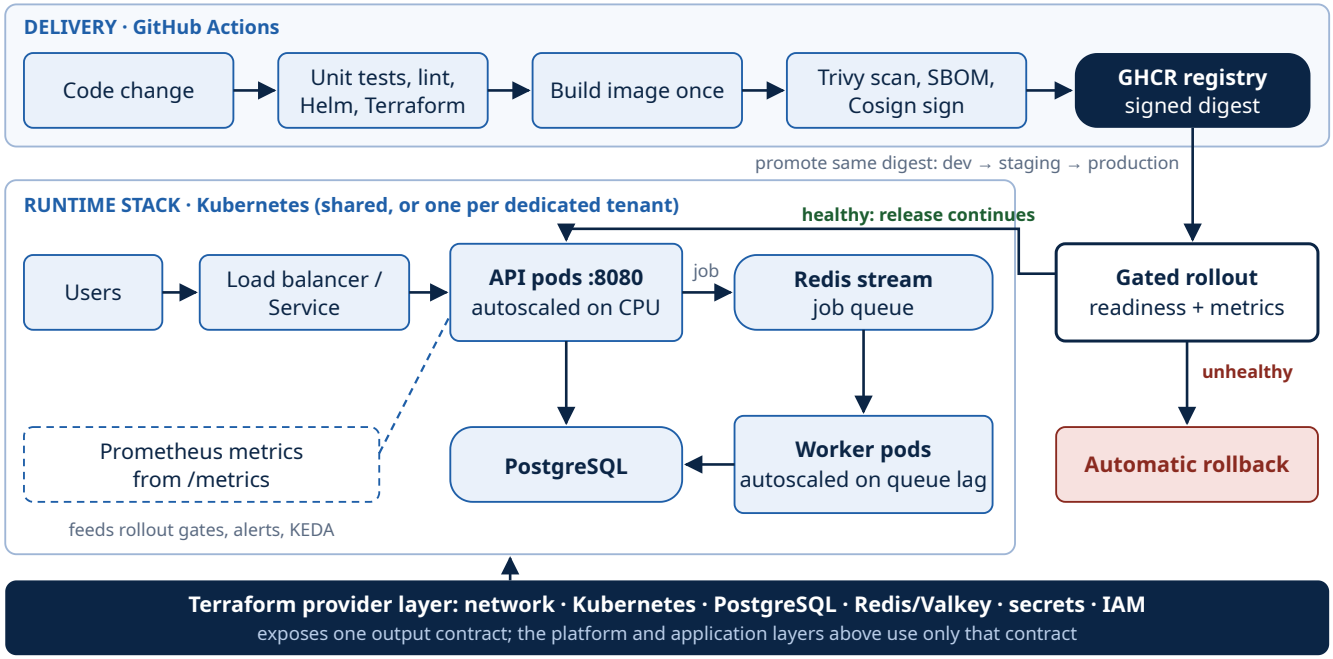


Figure 1. Main components and data flow. A release is promoted by digest and goes through the gated rollout; an unhealthy release is rolled back automatically. Dashed lines show monitoring.

3. Infrastructure as code

Structure. All Terraform is under `infra/`:

Root	Purpose
`bootstrap/aws-state`	Remote state bucket (encrypted, versioned, native locking) and GitHub OIDC roles for CI, so no long-lived cloud keys are stored
`modules/aws-stack`	Reusable provider module: VPC across 3 availability zones, EKS, PostgreSQL (Multi-AZ, PITR), Valkey (TLS, AUTH), KMS, Secrets Manager
`live/aws/shared-eu`	Shared multi-tenant stack in eu-central-1
`live/aws/dedicated-acme-eu`	Example dedicated tenant stack in eu-west-1, built from the same module
`providers/aws`, `providers/local`	Generic AWS root, and the local kind target with the same output contract
`platform`	Cluster add-ons for either target: metrics-server, KEDA, External Secrets, Argo CD/Rollouts, monitoring

Environments and tenants. Application settings for each environment are in `deploy/environments/` (local, dev, staging, production). Each tenant is a single values file in `gitops/tenants/` (examples: `shared`, `acme-eu`, `globex-eu`), and `gitops/fleet.yaml` defines the rollout rings. Dedicated tenants get their own infrastructure stack with a separate state file. EU residency is checked in Terraform: an EU tenant cannot be declared in a non-EU region.

Validation performed Tested, on 4 November 2026 (submission-validation.txt):

- `terraform fmt` clean across `infra/`;
- `terraform validate` passed for all 7 roots;
- `terraform test` passed **4 of 4** residency tests (mocked AWS provider);
- Helm lint passed, all 7 environment and tenant renders succeeded, and a server-side dry-run on the kind cluster succeeded.

Not done. `terraform plan` and `apply` were not run against a real account, so no cloud resources exist. A code review also found items to fix before any real apply (section 10).

4. CI/CD pipeline

The pipeline is defined in `.github/workflows/ci.yml` and `promote.yml`. Every third-party action is pinned to a full commit SHA, and the pipeline uses short-lived GitHub tokens only.

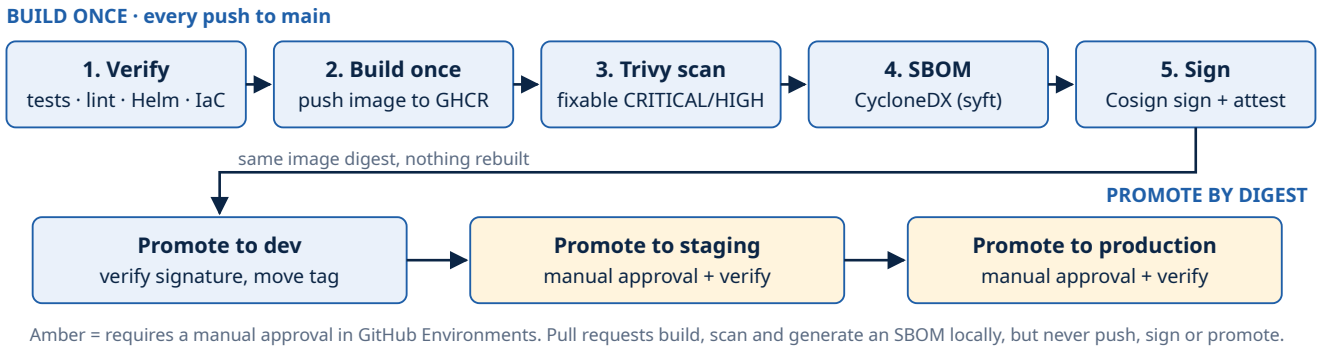


Figure 2. Pipeline stages. Promotion moves environment tags to the same image digest; nothing is rebuilt.

Stage	What happens
Verify	14 unit tests (Redis mocked), shell-script lint, Helm lint and render of every values file, Terraform format and validate
Build	Exactly one Docker build. The resulting image digest is the release identity for every later stage
Scan	Trivy 0.70.0 (container pinned by digest) fails the build on any fixable CRITICAL or HIGH vulnerability. A scanner error fails separately and is never reported as clean
SBOM	CycloneDX software bill of materials generated with syft
Sign	Keyless Cosign signature and SBOM attestation using the GitHub OIDC identity, so there is no signing key to leak
Promote	For dev, then staging, then production: verify the signature and attestation against this repository's workflow identity, point the environment tag at the **same digest**, and check that the promoted digest equals the built one. Staging and production require a manual approval

Final successful run Tested: GitHub Actions run #5 (41810479665), commit `9d747ff`, 4 November 2026.

- **Jobs:** all 5 successful.
- **Image:** digest `sha256:5bd2b6b4…db0e3`, promoted to dev, staging and production. Staging and production were each approved by the repository owner.
- **Independent re-check from the laptop:** the signature, SBOM attestation (CycloneDX 1.7, 2,813 components) and the binding of the signing certificate to commit `9d747ff` were re-verified with Cosign v3. Checks with the wrong identity failed, as expected.

What promotion means. Promotion moves registry tags. It does **not** deploy to a live Kubernetes cluster, because no cloud cluster exists. Two earlier runs (#1 and #2) failed because the scanner installer exited before scanning. The fix was to run the official Trivy container pinned by digest; the failures are kept on record.

5. Testing and evidence

All results below come from raw output files in `evidence/submission-2026-11-04/`. Every test ran on a **local single-node kind cluster** with the reference application. Load was generated by k6 running inside the cluster.

Test	Load	Requests	Failed	Other results	Status
Deploy under load	150 rps for 240 s	36,001	**0**	p95 latency 2.91 ms; ready API endpoints never below 5; rollout 11:19:46 → all 4 metric gates passed 11:22:24 IST	**Tested**
Rollback: readiness failure	100 rps	15,000	**0**	New pods never became ready; Helm rolled back automatically (revision 28 failed → 29); ready endpoints dipped to 4 of 5	**Tested**
Rollback: bad metrics	100 rps	20,000	**2,649 (13.2%)**	The release passed its probes but returned errors; the metric gate measured a 32.2% error rate and rolled back automatically (revision 26 → 25)	**Tested**
15x load (ratio)	10 → 150 → 10 rps over 365 s	35,099	**0**	p95 4.03 ms; API pods 2 → 5, worker pods 1 → 6 and back; **1,170 / 1,170 jobs completed**, 0 pending	**Tested** (ratio only)
Backup and restore	n/a	n/a	n/a	`pg_dump` → restore into a **new** database; schema, row count (2 = 2) and content checksum identical	**Tested**
Unit tests	n/a	14 tests	**0**	14 / 14 passed (locally and in CI)	**Tested**

How to read the bad-metrics result. The 2,649 failed requests are the expected cost of that test. The release was deliberately broken in a way the health probes could not see. The failures show how much traffic a bad release reaches before the metric gate removes it (about 87 seconds). This measured exposure is the main reason the design uses a 10% canary in the cloud.

Backup and restore detail. A custom-format `pg_dump` (SHA-256 `78fc3e4f…a2e44a`) was restored into `migration_demo_restore_20261104113132`. Before and after, the table structure, the 2 rows and the content MD5 (`f9c4e0a9…b8c8`) were identical. This proves the procedure and its verification on a small demo database. It does not measure restore time at production size.

Further evidence, also **Tested** locally:

- **Worker drain:** a 45-second job was interrupted by a restart. The old worker finished and acknowledged it, then exited after 41 s, inside its 75 s limit.
- **Tenant onboarding:** 16 s per tenant for the Kubernetes layer.
- **Fleet rollout:** a bad release failed at the first ring and the fleet rollout halted, leaving the other rings untouched.
- **Isolation and EU residency:** 9 of 9 checks passed.

Failed attempts are kept. Two early deploy-under-load attempts were rolled back by gate issues; both were fixed, and clients still saw 0 failures. There were also an invalid isolation control, a KEDA install timeout and a failed Helm upgrade. All are kept unchanged and listed in limitations.md.

6. Security and reliability

Control	How it works	Status
Vulnerability scanning	Trivy blocks any image with fixable CRITICAL/HIGH findings; nothing unscanned can be signed or promoted	**Tested** in CI
SBOM	CycloneDX SBOM attached to the image as a signed attestation	**Tested** in CI

Control	How it works	Status
Image signing	Keyless Cosign signature; every promotion re-verifies it against this repository's workflow identity	**Tested** in CI
Digest-based promotion	The bytes tested in staging are the bytes released to production; tags can't silently change what runs	**Tested** in CI
No stored cloud secrets	GitHub OIDC for CI; database password managed by the cloud provider; secrets synced by External Secrets	**Implemented**
Hardened containers	Non-root user, read-only root filesystem, all Linux capabilities dropped, default seccomp	**Tested** locally
Zero-downtime mechanics	Readiness probes, pre-stop delay, graceful shutdown, disruption budgets, no unavailable pods during rollout	**Tested** locally
Automatic rollback	Gate 1 (readiness) and gate 2 (error rate and latency from live metrics) both trigger `helm rollback`	**Tested** locally
Backup and restore	Restore always goes into a new database and is verified by schema, row count and checksum	**Tested** locally
Database migrations	Expand → backfill → contract, so each release stays compatible with the previous one	SQL demo **Tested**; app integration **Not done**

7. Architecture decisions and risks

Full detail is in the architecture document (5 pages): 8 decisions, each with its reason, rejected alternatives and trade-offs. The main ones:

Decision	Main reason	Alternative rejected
Kubernetes as the portable layer, with one Terraform module per cloud behind a shared output contract	Keeps cloud-specific parts visible; the same chart runs locally and in the cloud	One module with provider branches; a PaaS
Build once and promote by digest, with keyless signing	Production runs exactly what was tested; there is no signing key to leak	Rebuilding per environment; mutable tags
GitOps pull (Argo CD) for the cloud	CI never holds cluster credentials	Pipeline push with cluster credentials
Canary releases for the API, gated on metrics	A 10% canary limits the blast radius measured in the bad-metrics test	Blue/green (twice the capacity at peak); plain rolling
Rolling updates with drain for workers, scaled on queue lag	Queue backlog is the real demand signal for background jobs	CPU-based scaling; canary for workers
Dedicated tenants get separate infrastructure stacks	Independent failure domains for the 99.95% target and data residency	Namespace-only isolation

Top risk	Impact	Mitigation
R1. The real application's startup migrations are not backward compatible	Old and new versions conflict; rolling back code doesn't restore data	Expand/contract policy, migration job with a lock, snapshot before destructive steps, point-in-time restore
R2. Scale-up is too slow to reach 15x within 5 minutes	Higher latency and errors at peak	Aggressive autoscaling policy, spare node headroom; **must be load-tested at 2,250 rps in the cloud (not done)**

Top risk	Impact	Mitigation
R3. Database connections run out at peak (about 354 needed)	Errors for all tenants	Per-pod pool limits, connection cap, RDS Proxy/PgBouncer, alert at 80%
R4. Jobs lost or processed twice during failures	Lost or duplicated business work	Acknowledge only after completion, idempotency marker, reclaim of unacknowledged jobs
R5. EU data leaks through backups, logs or support tools	Contract or legal breach	Region checks in three layers, no cross-region copies for EU stacks, periodic audit

8. Additional change request

The brief included a change request, issued 24 hours into the assignment: a **second dedicated stack on a different cloud, with US-West as primary and US-East as failover**. The answer is a one-page addendum, CR-001.

Proposed design Design

- **Cloud and regions:** Google Cloud, with **us-west1** as primary and **us-east4** as a warm standby, for one dedicated US tenant. EU data is never replicated to it.
- **Infrastructure:** a new `gcp-stack` Terraform module exports the same output contract. The services used are GKE, Cloud SQL with high availability and a cross-region replica, Memorystore, Secret Manager and Cloud KMS. The platform and application layers are unchanged and run the same signed image.
- **Recovery targets:** RPO ≤ 1 minute and RTO ≤ 30 minutes. Failover is approved by a person, because a database split-brain is worse than a few minutes' delay. Failback has its own procedure.
- **Release requirement:** go-live is no earlier than **24 hours after change approval**, and only after a failover and failback game day has passed and the tenant owner has signed off.
- **Cost:** an estimated $2.0–2.1k per month. This takes the fleet slightly over the $9,000 steady budget unless it is offset or charged to the tenant.

Status. This is a **documented design only**. No second-cloud infrastructure was provisioned, tested or released.

9. Submission links

Item	Link
GitHub repository	github.com/Girish7010/cloud-agnostic-zero-downtime-platform
Successful GitHub Actions run #5	actions/runs/41810479665
Architecture document (PDF, 5 pages)	docs/pdf/architecture.pdf
Change request CR-001 (PDF, 1 page)	docs/pdf/change-request-us.pdf
Evidence index	evidence/submission-2026-11-04/README.md
Deploy-under-load evidence	deploy-under-load/
Rollback evidence (metrics)	rollback-bad-release-metrics/
Rollback evidence (readiness)	rollback-bad-release-readiness/
15x load-test evidence	load-15x-ratio/
Backup and restore evidence	backup-restore/
Limitations	limitations.md
Final submission report (requirement matrix)	FINAL_SUBMISSION_REPORT.md

10. Limitations and future improvements

Limitations, disclosed in full in limitations.md:

1. **Local environment only.** All runtime testing ran on a local **single-node Kubernetes kind cluster** on a laptop with 6.9 GiB of memory. The load generator ran on the same node, so the latency figures don't represent production.
2. **No cloud infrastructure.** No production AWS or GCP infrastructure was provisioned. The Terraform is validated and tested against a mocked provider only.
3. **Promotion is not deployment.** CI promotion to dev, staging and production moves registry tags. It does not deploy to a live Kubernetes production cluster.
4. **15x as a ratio.** The 15x test went from 10 to 150 rps. **The 2,250 rps peak in the brief was not tested**, because it needs cloud capacity and spend approval.
5. **Reference application.** The application was written to the black-box contract; the original black-box application was not supplied. Its startup database migrations were therefore not tested.
6. **No production zero-downtime claim.** "No errors during deployment" means 0 of 36,001 requests failed in one local test. It doesn't prove zero downtime under production conditions (multiple nodes, cloud load balancer, real application).
7. **Configuration-only components.** Argo CD, Argo Rollouts and Prometheus alerting are configuration only; they weren't installed locally because of memory limits.
8. **Estimates.** Cloud onboarding time (20–25 min) and costs are estimates, not measurements.
9. **Change request.** The second-cloud design is documented, not implemented or tested.

Future improvements, in priority order:

- Before any cloud apply, fix the code-review findings:
  - narrow the CI apply role's IAM permissions with a permissions boundary;
  - restrict the Kubernetes API endpoint to known networks;
  - narrow the plan role's trust and secret access;
  - add context checks and timeouts to the test scripts.
- Provision the shared EU stack, then run the 2,250 rps peak and canary analysis in the cloud.
- Integrate and test with the real black-box application, including its startup migrations.
- Implement CR-001 and run the failover and failback game day.

11. Conclusion

This submission delivers a working, reproducible platform design with clear evidence of what it does today. Locally, these are tested with raw evidence:

- zero-error deployment under load;
- automatic rollback of bad releases;
- queue-based autoscaling across a 15x load ratio without losing jobs;
- verified backup and restore;
- tenant onboarding, isolation and EU-residency controls.

The CI/CD pipeline builds once, scans, generates an SBOM, signs, and promotes the same image digest through approval-gated environments; this is proven by a successful GitHub Actions run. The cloud infrastructure and the second-cloud change request are implemented or documented to review standard and labelled clearly as not provisioned.

All four assignment deliverables are complete and ready for review. I am happy to walk through any part of the design, code or evidence in the technical interview.